

Implementation of the Dynamic Time Warping Algorithm for Bioacustic Detection

by

Elisabeth Christina Bosch

Dissertation presented for the degree of Electrical and Electronic Engineering in the Faculty of Engineering at Stellenbosch University

Supervisor: Prof. Jaco Versfeld

2026

Acknowledgements

* Supervisor X.
* Funding Y.

> *A very long quote*
> *from someone very important.*
> – Mr M

Thank you to Leanne for helping to prepare this template.

Declaration

By submitting this dissertation electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date: day October 2026

Copyright © 2026 Stellenbosch University
All rights reserved.

Abstract

Table of Contents

Abstract **iii**

Table of contents **iv**

List of figures **vi**

List of tables **vi**

Nomenclature **viii**

Acronyms and abbreviations viii

Definitions ix

1. Introduction **1**

1.1. Background 1

1.2. Problem Statement 2

1.3. Objectives 2

1.4. Scope 2

1.5. Layout of report 3

2. Literature **4**

2.1. Bryde's Whales 4

2.2. Dynamic Time Warping 5

2.3. Feature Extraction Methods 8

2.3.1. Short Time Fourier Transform 8

2.3.2. Mel-Frequency Cepstral Coefficients 9

2.3.3. Linear Predictive Coding Coefficients 10

2.4. Similar Studies 11

2.4.1. Classification of Killer Whale Vocalisations using DTW 11

2.4.2. Analysis of Template-Based Detection Algorithms for Bryde's Whales 12

2.4.3. Detection of real-time fin and blue whale calls 12

2.4.4. Hidden Markov Model for Bryde's Whale short pulse calls 13

2.4.5. Speech Recognition using DTW 13

3. Dataset **15**
3.1. Background . . . 15
3.2. Labelling . . . 15
3.3. Data Splitting . . . 15
3.3.1. Template Selection . . . 16
3.3.2. Hyperparameter Tuning Template Selection . . . 16
3.3.3. Evaluation Data Selection . . . 17

4. System Design **18**
4.1. Overview . . . 18
4.2. Feature Extraction Methods . . . 18
4.2.1. Short-Time Fourier Transform . . . 18
4.2.2. Mel-Frequency Cepstral Coefficients . . . 19
4.2.3. Linear Predictive Coding Coefficients . . . 19
4.3. DTW Methodology . . . 20
4.4. Hyperparameter Tuning . . . 21
4.5. Evaluation Procedure . . . 24
4.5.1. Detection Methodology . . . 24
4.5.2. Matching Methodology . . . 26
4.6. Evaluation Metrics . . . 27

5. Results **29**
5.1. Experiment 1: STFT Features . . . 29
5.2. Experiment 2: MFCC Features . . . 31
5.3. Experiment 3: LPCC Features . . . 35
5.4. General Observations . . . 37

6. Conclusion **40**

References **41**

A. Appendix title goes here **46**

List of Figures

2.1. Examples of how each call type looks: single-tone, multi-tone, burst-tonal and tonal downsweep. 4
2.2. An example of the optimal warping path between two vector features, X with length N = 4 and Y with length M = 6 5
2.3. An example for the three possible options for min as well as an example of the optimal warp path found for signals X and Y assuming X=[0, 2, 1, 0] and Y=[0, 0, 0.5, 2, 1, 0] 7
2.4. Example of a subsequence DTW alignment between two signals $X = [x_1, x_2, x_3, x_4]$ and $Y = [y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}]$ 7
2.5. The mel scale filter bank is made up of 20 to 40 triangular filters arranged according to the mel scale. 9

4.1. Software flow diagram describing the hyperparameter selection procedure. . . 21
4.2. Software flow diagram describing the evaluation procedure. 24
4.3. Illustration of how k-th lowest costs are extracted and scaled for a sample configuration with three templates per call type, two thresholds, and $k = 2$. . 25

5.1. The PR curves for STFT 31
5.2. The PR curves for MFCC 34
5.3. The PR curves for MFCC with the first derivative added 34
5.4. The PR curves for MFCC with the first and second derivatives added 35
5.5. The PR curves for LPC 36
5.6. Examples of random samples of FP and TP for which some FP could be unlabeled calls for the 10 template experiment for the STFT model. 38
5.7. This shows for each model where FP calls were detected in relation to labeled calls over the entire 72 hour audio file. 38

List of Tables

3.1. Dataset counts by date and category 16

5.1. Tabulation of thresholds as well as the respective F1-scores used to choose the best frame length. The average F1-score over all three thresholds per STFT experiment was also given for easier analysis. 29
5.2. Vote frac results for STFT for 5 and 10 templates 30
5.3. Final test results for STFT for 5 and 10 templates 30
5.4. Calibration variables chosen via highest F1 for 5 templates and 10 templates 32
5.5. Final vote results for 5 and 10 templates 32
5.6. Final test results for 5 and 10 templates 33
5.7. Calibration variables chosen via highest F1 for LPC for 5 templates and 10 templates 35
5.8. Vote frac results for LPC for 5 and 10 templates 35
5.9. Final test results for LPC for 5 and 10 templates 36
5.10. Overall highest results for F1 and AP per model 37

Nomenclature

Acronyms and Abbreviations

AP	
DCT	Discrete Cosine Transform
DTW	Dynamic Time Warping
FFT	fast Fourier Transform
FT	Fourier Transform
LPC	Linear Predictive Coding
LPCC	Linear Predictive Coding Coefficients
MFCC	Mel-Frequency Cepstral Coefficients
SNR	
STFT	Short Time Fourier Transform

Definitions

F

f1-score i

frame duration i

H

hamming window i

P

precision i

R

recall i

S

sampling frequency i

spectral leakage i

Chapter 1

Introduction

1.1 Background

Van Waerebeek et al. [1] reported indications since the late 1990s that whales have been struck by shipping vessels. Today this has become a larger concern, as vessel speeds and traffic have substantially increased. Bryde's whales are found to migrate in smaller regions instead of the usual large-scale migrations of other whale species. Faustmann et al. [2] report that such a whale colony in False Bay, South Africa has also been placed in possible danger due to fishing activities in their migration region.

Detection and classification systems for whale calls can be used to monitor the migration patterns of specific whale colonies throughout the year. The information gained from such analysis can help determine when vessel traffic should be diverted for the safety of a whale population. These types of systems have been implemented on different whale colonies, as discussed by Betz et al. [3], but the migration patterns of the False Bay colony have not yet been thoroughly documented. Due to the variations in call types between different colonies of some whale species, such as killer whales (Deecke et al. [4]) and blue whales (McDonald et al. [5]), models trained for other Bryde's whale colonies would not be completely trustworthy for the detection of calls within the False Bay Bryde's whale colony, as the universality of calls cannot be assumed. Therefore, a model trained specifically for the False Bay colony would be beneficial.

Brown et al. [6] showed that dynamic time warping (DTW) is an effective method for classifying killer whale vocalizations. Given the limited labeled data for the False Bay colony, DTW is considered as Herreilers et al. [7] found that it requires less input data than other detection algorithms.

1.2 Problem Statement

This project will determine whether DTW applied to frame-based feature vectors is able to effectively detect Bryde's whale calls within the False Bay region, as measured by precision, recall and the f1-score. Three feature extraction methods will be compared in terms of detection performance. These will be the short time fourier transform (STFT), mel-frequency cepstral coefficients (MFCC) and linear predictive coding coefficients (LPCC).

1.3 Objectives

1. Implement a DTW detection system using three different feature extraction methods (STFT, MFCC and LPCC), and tune the feature frame length, call type thresholds and vote fraction for each method based on f1-score.

2. Evaluate each system on the full 72 hour evaluation recording using 5 and 10 templates per call type, by calculating the precision, recall, f1-score and average precision (AP).

3. Compare the f1-score and AP results of the three feature extraction methods to determine which is best suited for detection, and whether the effectiveness of the DTW system is independent of the feature extraction method.

4. Determine whether DTW is an effective detection method based on the f1-score and AP results for the best feature extraction method.

1.4 Scope

Elements not considered within this project include the classification of call types after detection, the detection of tonal downsweep calls, detection algorithms other than DTW, feature extraction methods other than STFT, MFCC and LPCC, the accuracy of detected start and end times, the detection of frequencies above 500 Hz.

Call type classification was not considered due to discontinuities in how calls were labeled. These discontinuities arose due to a variety of human labelers for different sections of data and the presence of similar frequency components across different call types.

Tonal downsweep calls were excluded from the evaluation because labeled examples of this call type were only found in one of the three recordings.

Detections were evaluated on whether they overlap a labeled call, so the accuracy of the detected start and end times was not evaluated. Frequencies above 500 Hz were not considered, since the audio was downsampled to 1 kHz.

1.5 Layout of report

* Chapter 2 provides the background information needed to understand the problem, as well as literature on previous solutions to the problem.
* Chapter 3 provides background information on the dataset used for this project and explains how the data was split.
* Chapter 4 gives a detailed design of the entire project, including the feature extraction methods, the DTW implementation, hyperparameter tuning, the evaluation procedure and the evaluation metrics.
* Chapter 5 presents the results obtained from the designed system, as well as a description of what the results show.
* Chapter 6 contains the concluding thoughts on the results, including whether the objectives were met, as well as improvements and recommendations for the project.

Chapter 2

Literature

2.1 Bryde's Whales

Bryde's whales predominantly live in tropical or subtropical environments across the world. The different call types, rates, frequencies and harmonics of these whales vary depending on the geographical location of the colony. Generally Bryde's whales have calls containing frequencies from as low as 42 Hz to as high as 1 kHz [8].

In the dataset used for this project there were four main call types present. These are referred to as single-tones, multi-tones, burst-tonals and tonal downsweeps.

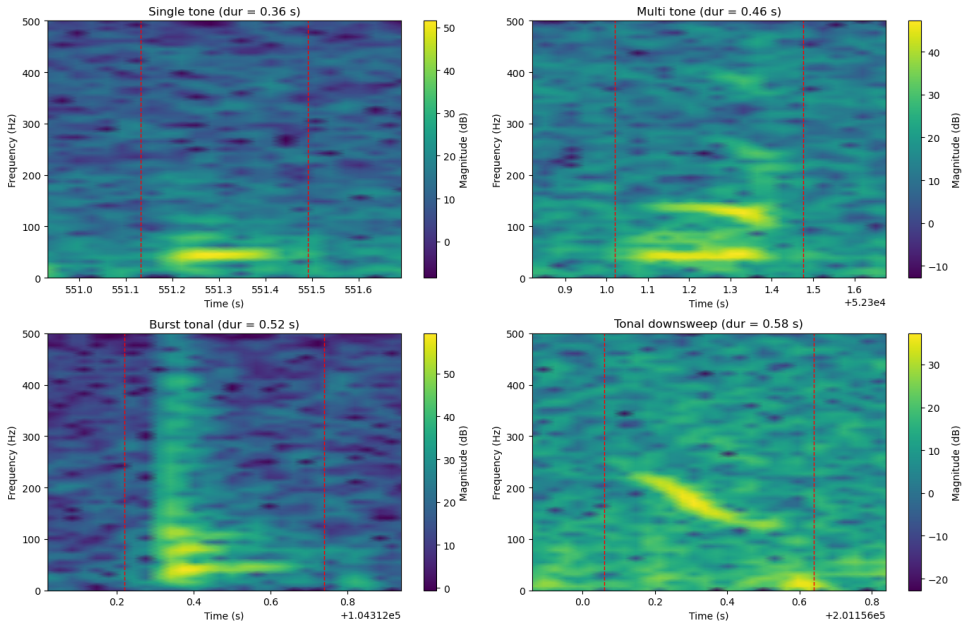


Figure 2.1: Examples of how each call type looks: single-tone, multi-tone, burst-tonal and tonal downsweep.

The single-tone calls were characterised by a singular frequency at around 42 Hz. Multi-tones also contained the 42 Hz frequency, but there were one or two other frequencies above this with a maximum frequency component at about 200 Hz. Burst-tonals also have the multiple frequency characteristic of the multi-tones, however the lower frequencies, 42 - 200 Hz, were held for longer than the higher frequencies, 200 - 500 Hz. The tonal downsweep was the only call type not characterised by the 42 Hz frequency. It was a sweeping of frequencies through time starting at around 200 Hz and ending at around 100 Hz [2].

In accordance with the original detected calls the average call length of each call type was around 0.3 seconds with a maximum duration of about 2.5 seconds. In other studies of Bryde's whale calls in different geographical areas, a maximum duration of around 7.0 seconds was identified by Faustmann et al. [2] and Oleson et al. [8].

Decisions made in the project pertaining to frequencies and durations falling within the detection limits were made based on the above information.

2.2 Dynamic Time Warping

Dynamic time warping (DTW) is an algorithm for detecting the similarity of two signals, $X = [x_1, x_2, \ldots, x_n]$ and $Y = [y_1, y_2, \ldots, y_m]$. This measure of similarity is achieved by calculating an optimal alignment cost between the two signals of length $N \in \mathbb{N}$ and $M \in \mathbb{N}$ respectively. In terms of feature vectors such as the short time Fourier transform (STFT), mel-frequency cepstral coefficients (MFCC) or linear predictive coding coefficients (LPCC) this means that each frame making up one signal is compared to each frame making up another signal. Using an alignment cost the best alignment for each frame in one signal to the other signal is chosen and an overall DTW cost is calculated to show how similar the overall signals are. Figure 2.2 shows, as explained by Müller and Zalkow [9] and Kamper [10], that such an optimal alignment path may mean that a single frame in signal X may produce the lowest cost against more than one frame in signal Y and vice versa.

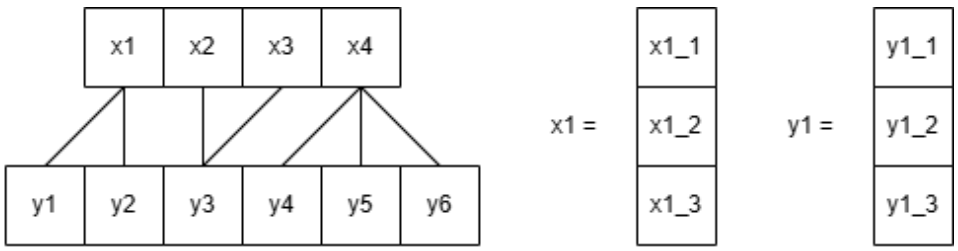


Figure 2.2: An example of the optimal warping path between two vector features, X with length N = 4 and Y with length M = 6

The overall cost needed to determine how similar the two signals are is calculated via a

cost matrix. Cost matrix, $C_{N+1,M+1}$ for signals X and Y, is initialised such that $C_{i,0} = \infty$ for i = 1 to N, $C_{0,j} = \infty$ for j = 1 to M and $C_{0,0} = 0$.

The formula used to fill each cell of the cost matrix is:

$$C_{i,j} = \text{dist}(x_i, y_j) + \text{Q} \qquad \text{with} \quad \text{Q} = \min \begin{cases} C_{i-1,j-1} & \text{(match)} \\ C_{i-1,j} & \text{(insertion)} \\ C_{i,j-1} & \text{(deletion)} \end{cases} \tag{2.1}$$

where $dist(x_i, y_j)$ represents the difference between X_i and Y_j as a distance. For vector signals this distance is calculated between each frame i of signal X and each frame j of signal Y. According to Kamper [10], this can be done using various different techniques including cosine, Euclidean or dot product similarity. For this project only the cosine distance and Euclidean distance metrics were used.

Euclidean distance is sensitive to the magnitude and direction of the vectors for which it is calculated. In basic terms, as described by Schwaber-Cohen [11], it is the absolute difference between two vectors.

$$d(\mathbf{x_i}, \mathbf{y_j}) = \sqrt{(\mathbf{x_i}_1 - \mathbf{y_j}_1)^2 + (\mathbf{x_i}_2 - \mathbf{y_j}_2)^2 + \cdots + (\mathbf{x_i}_n - \mathbf{y_j}_n)^2} \tag{2.2}$$

Cosine similarity is only sensitive to the direction of the vectors for which it is calculated. In basic terms, as described by Schwaber-Cohen [11], it is the angle between the two vectors.

$$\cos(\mathbf{x_i}, \mathbf{y_j}) = \frac{\mathbf{x_i} \cdot \mathbf{y_j}}{\|\mathbf{x_i}\| \, \|\mathbf{y_j}\|} \tag{2.3}$$

Since DTW minimises the cost between frames, the cosine similarity cannot be used directly as the distance, because a higher similarity must give a lower cost. Instead the cosine distance is used, which is defined in the SciPy documentation [12] as:

$$d_{\cos}(\mathbf{x_i}, \mathbf{y_j}) = 1 - \cos(\mathbf{x_i}, \mathbf{y_j}) \tag{2.4}$$

A cosine distance of 0 therefore means that the two vectors point in the same direction.

Q is the minimum value of three possible options namely match, insertion or deletion. Match is the value in the matrix cell diagonally down and left from the current cell for which the cost is being calculated. Insertion is the value in the matrix cell directly below the current cell and deletion is the matrix cell directly to the left of the current cell.

The cost which is then calculated for the current cell, $C_{i,j}$, is a summation of the distance between the feature frames i and j in signals X and Y respectively as well as the lowest accumulated cost up until that point. This means that the final cell reached within the cost matrix, $C_{N+1,M+1}$, will contain the lowest overall cost for the similarity of signals X and Y. The lower the final cost, the higher the similarity between the two signals.

The example shown in Figure 2.3 assumes that X and Y are scalar signals, however, as explained by Kamper [10], the same logic is used when X and Y are vectors.

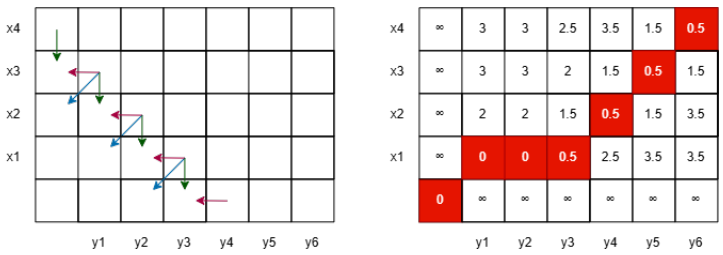


Figure 2.3: An example for the three possible options for min as well as an example of the optimal warp path found for signals X and Y assuming X=[0, 2, 1, 0] and Y=[0, 0, 0.5, 2, 1, 0]

The general case of DTW finds global alignment between the two signals. This alignment is found using the warping path as described by Alizadeh [13]. The warping path is the path of the best alignment between signals X and Y within the cost matrix, as shown in Figure 2.3. This path, W, is a sequence of matrix co-ordinates, (i, j), with a total length k.

The boundary condition for global alignment ensures the final path starts and ends with the start and end frames of each respective signal.

$$i_1 = 1, i_k = N \quad \text{and} \quad j_1 = 1, j_k = M \tag{2.5}$$

The monotonicity condition ensures that each new point in the warping path is a step forward in time from the previous step.

$$i_{t-1} \leq i_t \quad \text{and} \quad j_{t-1} \leq j_t \tag{2.6}$$

The continuity condition ensures the step size of the warping path is only one unit in time.

$$i_t - i_{t-1} \leq 1 \quad \text{and} \quad j_t - j_{t-1} \leq 1 \tag{2.7}$$

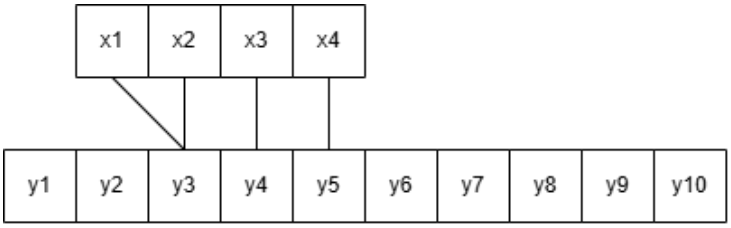


Figure 2.4: Example of a subsequence DTW alignment between two signals $X = [x_1, x_2, x_3, x_4]$ and $Y = [y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}]$

Subsequence DTW is a version of the DTW algorithm that alters the boundary condition such that global alignment is no longer necessary. Instead the entirety of signal X is now matched with a subsequence of signal Y. The new boundary condition allows the warping path to start and end at any point within the first and final row of the matrix respectively.

$$i_1 = 1,\ i_k = N \quad \text{and} \quad 1 \leq j_1 \leq j_k \leq M \tag{2.8}$$

In global alignment the final cell in the cost matrix, $C_{N+1,M+1}$, is considered the overall optimal cost for those two signals. As described by Müller and Zalkow [14], with subsequence DTW the entire final row of the matrix is checked to find the lowest cost. The position of that cost shows where the end point of the alignment detected via subsequence DTW is.

2.3 Feature Extraction Methods

2.3.1 Short Time Fourier Transform

According to Usman et al. [15] and Kamper [16], the short time Fourier transform (STFT) is used to analyse the change in frequency of a signal over time. The STFT of a segment of audio is calculated by multiplying the signal in the time domain, $x(t)$ with a window $w(t-\tau)$ which is swept across said signal. The Fourier transform (FT) of each window is calculated and used to determine the frequency information for that window. Thus, the STFT provides the change in frequency for a segment of audio throughout time.

The mathematical formulation of STFT is:

$$X(\tau, \omega) = \int_{-\infty}^{\infty} x(t)\, w(t-\tau)\, e^{-j\omega t}\, dt, \tag{2.9}$$

where τ is the position of the window as it shifts through time at step-size H and ω is the frequency being analysed.

The frequency resolution of the STFT is given as:

$$resolution = \frac{f_s}{L} \tag{2.10}$$

where f_s is the sampling frequency of the signal and L is the length of the window over which the FT is taken.

This means that the STFT has a trade-off between time and frequency resolution.

If L is kept short, time resolution is high since smaller segments of time are being used for one FT analysis. However, frequency resolution is then decreased since too little time passes for the FT to distinguish between frequencies spaced closer together.

If L is kept long, time resolution drops. This means that fast changes in frequency will be missed. However, frequency resolution increases and it becomes easier to distinguish between different frequency bands.

Window overlap is another consideration when using STFT. Increased overlap creates a smoother representation of the changes in frequency occurring over time since information at the edges of the window is no longer lost. The trade-off with overlap is the increase in computational complexity that will occur due to smaller and thus more windows of a signal being analysed.

2.3.2 Mel-Frequency Cepstral Coefficients

Mel-frequency cepstral coefficients (MFCC) is a common choice for feature extraction when working on acoustic data. This is because, as explained by Kamper [17], it is set up to perceive frequencies as they are perceived by the human ear. That means the lower frequencies are given a higher resolution than the high frequencies. Features are initially extracted using the fast Fourier transform (FFT). After this they are passed through a mel-scale filter bank. The formula for the mel-frequency calculation given by Usman et al. [15] is:

$$Mel(f) = 1127 \ln\left(1 + \frac{f}{700}\right) \tag{2.11}$$

with $Mel(f)$ as the frequency value on the mel scale and f as the actual frequency in Hz.

This bank is made up of 20 to 40 triangular filters which are placed, smallest width to largest width according to the mel scale, along the frequency spectrum. According to Singh [18] and Kamper [17], the standard amount of triangular filters chosen is 26. This results in 26 coefficients for the entire range of frequencies being analysed.

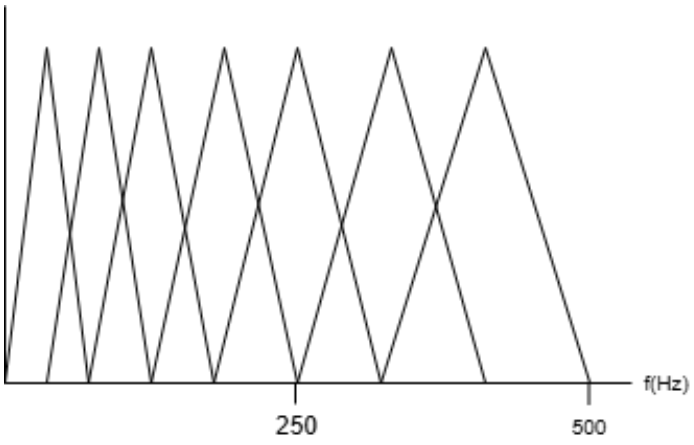


Figure 2.5: The mel scale filter bank is made up of 20 to 40 triangular filters arranged according to the mel scale.

The log of these coefficients is then taken and finally the discrete cosine transform (DCT) is used to decorrelate the filter bank coefficients. That means any patterns linking different coefficients to each other are removed such that the coefficients become independent. The final number of cepstral coefficients returned by the DCT is typically 12 or 13 since the most important information is kept there. The formula used by Usman et al. [15] to perform the DCT is:

$$MFCC_i = \sum_{k=1}^{K} X_k \cos\left(\frac{i(k-0.5)\pi}{K}\right), \quad i = 1, 2, \ldots, K \tag{2.12}$$

with i as the index of the current coefficient, K as the total number of triangular filters used, k as the specific filter bank at that point in time and X_k as the energy after log was applied to the mel coefficients.

As explained by DataMListic [19][20], the first and second derivatives, often referred to as deltas, of the cepstral coefficients are often used in feature extraction. They show the change of each cepstral coefficient across frames. This can be vital information when analysing how frequency changes across time since the MFCC process tends to lose this information. The formula for the derivatives, as given by Majeed et al. [21], is:

$$x_t^{(d)} = \frac{\sum_{m=-M}^{M} m \cdot x_{t+m}^{(d-1)}}{\sum_{m=-M}^{M} m^2} \tag{2.13}$$

with t as the specific point in time, d as the order of derivative, M as the number of frames on either side of t and m as the frame offset.

2.3.3 Linear Predictive Coding Coefficients

As explained by Rabiner and Juang [22] and Usman et al. [15], linear predictive coding (LPC) is a straightforward mathematical method commonly used for feature extraction. LPC focuses on the method of approximating a sample of a speech signal using a combination of p samples from the past. This means that for each sample for which there are p available samples before it an approximation is made in the form of the equation:

$$\tilde{s}(n) = \alpha_1 s(n-1) + \alpha_2 s(n-2) + \cdots + \alpha_p s(n-p) \tag{2.14}$$

where $\tilde{s}(n)$ is the approximated speech sample and $\alpha_1, \alpha_2, \ldots, \alpha_p$ are the coefficients for that approximation. The p coefficients used are constant over all samples within the given frame.

For each sample with an approximation an error is calculated between the real sample and the approximated sample.

$$e_n = s(n) - \tilde{s}(n) \tag{2.15}$$

The sum of the squares of all these errors is minimised to find the p constant coefficients. The equation for this is:

$$E_n = \sum_m e_n^2(m) \tag{2.16}$$

where m is the sample for which the error $e_n(m)$ was calculated.

The feature extracted via this method is made up of the p LPC coefficients which were calculated to minimise the mean square error. These coefficients tend to be unstable, therefore cepstral coefficients can be calculated from these raw LPC coefficients giving linear predictive coding coefficients (LPCC). LPCC gives the coefficients of the Fourier transform (FT) representation of the magnitude spectrum and is more reliable than the LPC representation.

LPCC can be calculated from the LPC results through recursion. The formula for that is:

$$\begin{aligned} c_0 &= \ln \sigma^2 \\ c_m &= a_m + \sum_{k=1}^{m-1} \left(\frac{k}{m}\right) c_k a_{m-k}, \qquad 1 \leq m \leq p \\ c_m &= \sum_{k=1}^{m-1} \left(\frac{k}{m}\right) c_k a_{m-k}, \qquad m > p, \end{aligned} \tag{2.17}$$

where σ^2 is the LPC gain term.

The final feature extracted using LPCC contains p coefficients for a single frame, with the zeroth coefficient, c_0, containing the magnitude information.

2.4 Similar Studies

2.4.1 Classification of Killer Whale Vocalisations using DTW

An article, "Automatic classification of killer whale vocalizations using dynamic time warping", written by Judith C. Brown and Patrick J. O. Miller [23] evaluated four different variations of the dynamic time warping (DTW) algorithm to classify biphonic killer whale vocalisations using fundamental frequency contours.

Due to the biphonic nature of these calls the low-frequency component and high-frequency component for each call was extracted for separate analysis. Four different variations of the DTW algorithm with various different constraints were used to evaluate against each other.

The final result of the paper was that the method referred to as "Sakoe-Chiba" on the LPC performed the best with a 90% agreement with the labels selected by human

classifiers.

This study did show that the use of DTW in bioacoustic analysis is a good option. However, it did not consider the ability of DTW to detect calls in a continuous segment of bioacoustic data as all calls which were classified had already been selected and segmented to the pitch contours of the vocalisation. This study also did not consider the impact of low frequency analysis using DTW as killer whale vocalisations do not tend to fall below 500 Hz.

2.4.2 Analysis of Template-Based Detection Algorithms for Bryde's Whales

The article, "Analysis of Template-Based Detection Algorithms for Inshore Bryde's Whale Short Pulse Calls" written by Olayinka O. Ogundile and Daniel J. J. Versfeld [24] analysed two template based detection algorithms to detect short pulse calls for Bryde's whales in False Bay, South Africa. The two methods tested were dynamic time warping (DTW) and linear predictive coding (LPC). In their analysis they used two separate sample sets with different signal-to-noise ratio (SNR) values to also evaluate the performance of these techniques in the presence of noise. The input features for these models were the time-domain representation of the signals.

The final results of the article found that both LPC and DTW performed well in high and low SNR segments. LPC performed the overall best in low noise segments.

This article provides insight into the use of template based detection systems for Bryde's whales. This is relevant since DTW which is handled in this article is the chosen detection mechanism for this project. While LPC is also used in the article, its use is vastly different to the use within this project. LPC is used as a detection algorithm and tested on its ability to act as such. In this project LPC is not used as a detection algorithm, but instead linear predictive coding coefficients (LPCC) is used as a feature extraction method. The detection method focuses on the error component of the LPC equation while the feature extraction method focuses on the coefficients calculated for the approximation of the real signal. Another difference between this project and the article is that this article exclusively uses the time domain representation of the signal as input.

2.4.3 Detection of real-time fin and blue whale calls

The article, "Automatic recognition of fin and blue whale calls for real-time monitoring in the St. Lawrence" by Xavier Mouy, Mohammed Bahoura and Yvan Simard [25] tested detection models on real-time data for fin and blue whales. The detection methods used were dynamic time warping (DTW) and vector quantisation (VQ). This article focused on low frequency whale calls.

Instantaneous frequency, speed and acceleration were extracted as features. It was found that due to low frequency components from background elements such as ships the detection of false positives (FP) increased.

In terms of this project the article shows how low frequency detections can be affected by low frequency background noise. This article also considered real-time audio detection which is much closer in similarity to detection within a large audio file such as what was done in this project. This article did not use the same feature vectors as the project and did not consider their effect on DTW performance.

2.4.4 Hidden Markov Model for Bryde's Whale short pulse calls

The article "A hidden Markov model with selective time domain feature extraction to detect inshore Bryde's whale short pulse calls" by O.O. Ogundile, A.M. Usman, O.P. Babalola and D.J.J. Versfeld [26] compares a time domain feature extraction technique used for hidden Markov models (HMM) against mel-frequency cepstral coefficients (MFCC) and linear predictive coding (LPC), frequency domain techniques. The model is used for detection of Bryde's whale calls in False Bay, South Africa.

These three HMM models are used on 2 hours of continuous recordings to detect these calls. This is similar to the continuous recording detection done in this project.

The time domain feature HMM model performed the best with MFCC performing second best and LPC performing the worst. It was found that the MFCC and LPC models were sensitive to background noise.

While this does test detection on a continuous segment of audio, this article focuses on HMM detection and not DTW. It also tests the MFCC feature extraction method as well as the LPC method. Considering the fact that the LPC features were not converted to linear predictive coding coefficients (LPCC) this does not fully align with this project.

2.4.5 Speech Recognition using DTW

The goal of the article "Speech recognition using Dynamic Time Warping (DTW)" by Yurika Permanasari, Ermin H. Harahap and Erwin Prayoga Ali [27] was to analyse the effectiveness of dynamic time warping (DTW) in matching speech patterns. Variabilities were included in the study via pronunciation timing, volume and speaker identity.

Features were extracted from the recorded audio. Some audio was used as templates and others as test data. DTW was then used to test which of the variations impacted DTW distances the most. The main tests were keeping the pronunciation time the same with the same volume, different pronunciation time with the same volume and same pronunciation time with different volume.

It was determined that DTW was an effective pattern recognition algorithm and that pronunciation timing had a larger impact on DTW alignment than volume.

This study also uses DTW to recognise patterns in audio, however this focuses on speech recognition and the effects of variability on that recognition. It does not focus on different feature vectors or detection of low frequency whale calls in a continuous recording.

Chapter 3

Dataset

3.1 Background

The dataset used contains three 72-hour recordings, recorded in False Bay, near Gordon's Bay, between 22 April 2023 and 22 June 2023. The recordings were taken at a depth of 20 to 30 metres at the coordinates S 34.17°, E 18.76°.

The device used for the recordings was a single omnidirectional hydrophone, designed and built by a research group in the Department of Electrical and Electronic Engineering at Stellenbosch University.

The recorded data has a sampling rate of 8 kHz and a 16-bit resolution. The data was extracted as a 72-hour WAV file from an SD card used by the recording device [28].

3.2 Labelling

Detection tables, containing the start and end times of calls within each of the three files, were provided by a previous research project that used a band-limited energy detector to flag calls. This method unfortunately did not detect all the calls within the audio files, and none of the calls were labelled as a specific call type [28].

Another detection table was procured from Tonitz et al. [29], which contained more detections for the first 72-hour file and call labels for all three files. This meant that, while all three files contained labelled calls, only one contained enough labelled calls to use the entire file without expecting an excessive number of unlabelled calls.

3.3 Data Splitting

For this project the data was split into three sections: template data, hyperparameter tuning data and evaluation data. Each of the 72-hour audio files was assigned to a different

section. Given that only the first of the three 72-hour audio files was well labelled, it was decided that this file would be used as the evaluation data. The second audio file was used for hyperparameter tuning and the third was used to select templates. Labelled calls for the tonal downsweep call type were only found in the audio file used for evaluation. Therefore, due to a lack of data, this call type was ignored.

Table 3.1: Dataset counts by date and category

Recording Start Date	Call Count				Total Calls
	Single-Tone	Multi-Tone	Burst-Tonal	Tonal Downsweep	
22 April 2023	1015	452	171	164	1802
26 April 2023	242	91	9	0	342
29 April 2023	32	25	11	0	68

3.3.1 Template Selection

The audio file recorded starting on 29 April 2023 was used to select templates. These templates were used as the baseline for DTW comparisons.

For the three considered call types (single-tones, multi-tones and burst-tonals), the maximum number of templates per call type that could be extracted, such that each call type had the same number of templates, was 10. The limiting call type was burst-tonals. In Table 3.1 it can be seen that there were 11 available burst-tonals. However, it was decided to make 10 the maximum number of templates per call type.

To ensure that the 10 templates chosen were representative of varying SNR values, templates were chosen by looking at spectrograms of labelled calls throughout the audio file. The visual inspection was also necessary since some of the calls labelled as single-tones looked more like multi-tones and some of the multi-tones could actually be classified as burst-tonals. This overlap in possible classification was due to the variety of human labellers and the common frequency components between call types.

3.3.2 Hyperparameter Tuning Template Selection

The audio file recorded starting on 26 April 2023 was used to select templates used for hyperparameter tuning, referred to as call type calibration templates. These call type calibration templates were used to determine the DTW cost threshold below which a segment of audio could be considered a call. For the call type calibration templates, 10 templates were chosen for each call type. This was again limited by the maximum number of burst-tonal templates. In this file there were only 9 labelled burst-tonals. However, when selecting multi-tone templates, one call originally labelled as a multi-tone was judged,

based on its spectrogram, to be a burst-tonal. This call was relabelled and used as the tenth burst-tonal template, so that each call type had 10 call type calibration templates. The call counts in Table 3.1 reflect the original labels, before this change.

Besides these 30 call type calibration templates (10 for each call type), 100 background calibration templates were also selected. Within each audio file, most of the time was made up of background noise and only some parts actually contained calls. This was simulated by selecting many more background calibration templates than call-type calibration templates. Since there is no table labelling background noise, background calibration templates of varying sizes were extracted from the file and 100 templates were manually selected. The reason for the manual selection process is linked to the unreliability of the labelled data. The audio file used for selection of calibration templates was only labelled by the band-limited energy detector and thus did not label all the calls within the file. This means that some of the samples extracted as background audio actually contained calls that had never been labelled. Manually selecting each template ensured that all the templates chosen contained only background noise.

3.3.3 Evaluation Data Selection

The audio file recorded starting on 22 April 2023 was used for the final evaluation of the whale call detection model. No templates were extracted; instead, the entire file was swept to search for calls.

As seen in Table 3.1, this file was the only file that contained tonal downsweep calls. Due to the limited number of these calls, all the labelled tonal downsweeps within this file were ignored while testing.

Chapter 4

System Design

4.1 Overview

The project was coded in Python. The functionality of each section of the project was placed in relevant Python scripts. These were imported into the relevant Jupyter notebook used for each section.

4.2 Feature Extraction Methods

The raw audio recorded by the hydrophone was stored in WAV files with a sampling frequency of 8 kHz. Before extracting feature vectors from the recordings, the audio was first downsampled to a 1 kHz sampling rate. This helped remove unnecessary frequency information, as Meddins [30] explains that the Nyquist frequency ($\frac{f_s}{2}$) for a 1 kHz sampling rate is 500 Hz.

Faustmann et al. [2] report that most Bryde's whale vocalisations occur below 500 Hz. Thus, this sampling rate also ensures that no relevant information was lost prior to analysis.

4.2.1 Short-Time Fourier Transform

The `scipy.signal.stft` function from the SciPy library was used to calculate the short time fourier transform (STFT) feature vectors. The window length L, as referenced in Eq. (2.10), is calculated by multiplying f_s, the sampling of the given audio segment, by the chosen STFT frame duration. No fixed frame duration was pre-selected, as the optimal frame duration was determined during the hyperparameter selection stage of this project.

The window overlap H, as referenced in Eq. (2.10), is calculated by multiplying L by the chosen overlap percentage. For this project, the overlap percentage was set to 0.75. This high overlap percentage produced a smooth spectrogram when viewing the time-frequency representation of a given audio segment. A hamming window was chosen

to reduce spectral leakage caused by the finite length of the windows used to extract the feature vectors, as described by Kamper [16] and in the windowing functions guide [31].

4.2.2 Mel-Frequency Cepstral Coefficients

The `librosa.feature.melspectrogram` and `librosa.feature.mfcc` functions from the Librosa library, as documented by the Librosa development team [32], were used to calculate the cepstral coefficients for the mel-frequency cepstral coefficients (MFCC) feature extraction implementation. The window length required for the fourier transform (FT) is calculated by multiplying f_s by the chosen frame duration, which is optimally selected during hyperparameter tuning. Unlike the STFT function, `librosa.feature.mfcc` requires the hop length rather than the overlap length. The hop length is calculated by subtracting the window overlap, H, from the frame length, L. Here, H is calculated by setting the overlap percentage to 0.75 and multiplying it by L. A Hamming window is used to reduce spectral leakage.

The Librosa function used to calculate the MFCC features automatically applies discrete cosine transform (DCT) type 2 and ortho-normalisation, as described in the SciPy documentation [33]. These settings mean that the bottom row of the output matrix, c_0, represents the sum of the log-spectrum energy. Roch et al. [34] note that it is common practice in speech analysis to remove coefficient 0, as it contains the total signal energy and can dominate distance calculations in subsequent analysis. For this project, only coefficients 1 to 14 were retained. This ensured that the analysis considered only the overall shape of the log spectrum and not the total energy of the signal. The number of mel-spectrogram filters was set to 26, which is standard practice in audio processing. A total of 14 final coefficients were extracted (since 13 are typically used, retaining 14 allows 13 coefficients to remain after removing c_0).

The `librosa.feature.delta` function, as documented by the Librosa development team [35] and described by McFee et al. [36], was used to calculate the first and second derivatives for the MFCC features. The feature vector returned when using the first derivative was a concatenation of the original MFCC feature vector with its first derivative. This resulted in a feature vector with 26 coefficients (the 13 original coefficients plus 13 derivative coefficients). Similarly, for the second derivative, the concatenation of the original MFCC and first derivative was further concatenated with the second derivative, yielding a 39-coefficient feature vector.

4.2.3 Linear Predictive Coding Coefficients

The `librosa.util.frame` function [37] and the `librosa.lpc` function [38] from the Librosa library were used to calculate the linear predictive coding (LPC) features. The STFT and MFCC functions frame the audio internally; therefore, framing prior to feature

extraction was not required for those methods. However, the LPC function accepts a sequence of pre-framed audio segments to calculate the approximation coefficients. The `librosa.util.frame` function was used with a frame duration selected during hyperparameter tuning and a hop length of 0.25 of the frame duration ($1 - 0.75$). A Hamming window was applied to the frames before calling the `librosa.lpc` function.

Rabiner and Juang [22] explain that a higher LPC order yields a more detailed spectral approximation. However, considering that background noise could adversely affect coefficient calculations if the model becomes overly detailed, an order of $p = 4$ was selected.

Due to the numerical instability of raw LPC parameters, the LPC features were converted to linear predictive coding coefficients (LPCC). This was implemented using the recursive formula in Eq. (2.17). According to Rabiner and Juang [22], the number of LPCC coefficients typically selected is approximately $\frac{3}{2}p$. For $p = 4$, approximately 6 LPCC coefficients would standardly be calculated. After testing different numbers of coefficients, 8 LPCC coefficients were found to yield the best f1-scores during hyperparameter selection. Therefore, 8 LPCC coefficients were used.

4.3 DTW Methodology

The `librosa.sequence.dtw` function from the Librosa library, as documented by the Librosa development team [39], was used to calculate the DTW cost matrix between a template and an audio segment. The pairwise distance matrix required as input to this function was computed using the `scipy.spatial.distance.cdist` function from the SciPy library, as described in the SciPy documentation [12].

The distance function received the template feature and the audio segment feature extracted via STFT, MFCC, or LPCC, along with the specified distance metric. The two distance metrics evaluated were cosine distance and Euclidean distance. As shown in Eq. (2.3) and Eq. (2.2), cosine similarity calculates the angular difference between two vectors, whereas Euclidean distance calculates the straight-line distance between them. For the STFT feature extraction method, cosine distance was selected rather than Euclidean distance because cosine distance is invariant to magnitude. Consequently, the distance between a template and a distant (softer) call versus a nearby (louder) call depends solely on spectral shape rather than signal amplitude. This ensured that calls of varying loudness could be matched based on spectral properties.

For the MFCC implementation, signal energy variations due to distance from the recording device were mitigated by removing c_0, as described in Section 4.2.2. Similarly, for LPCC, signal energy is captured by the zeroth coefficient c_0, which is fixed to 1 during LPC extraction and excluded from the 4 primary LPC coefficients. As a result, Euclidean distance could be used without interference from signal amplitude, as only the spectral envelope remains. Because Euclidean distance is standard for DTW applications and

cosine distance provided no extra benefit for amplitude-normalised features, Euclidean distance was chosen for MFCC and LPCC.

The `subseq` parameter in `librosa.sequence.dtw` was set to `True`. As described in Section 2.2, subsequence DTW relaxes boundary constraints so that the start and end points of a detected match do not need to align with the boundaries of the candidate sequence. When scanning 1-hour segments of a 72-hour audio file, subsequence DTW enables the detection of multiple calls within a single segment by identifying dips along the final row of the cost matrix and selecting the local minima as call end points. This is detailed further in Section 4.5.

4.4 Hyperparameter Tuning

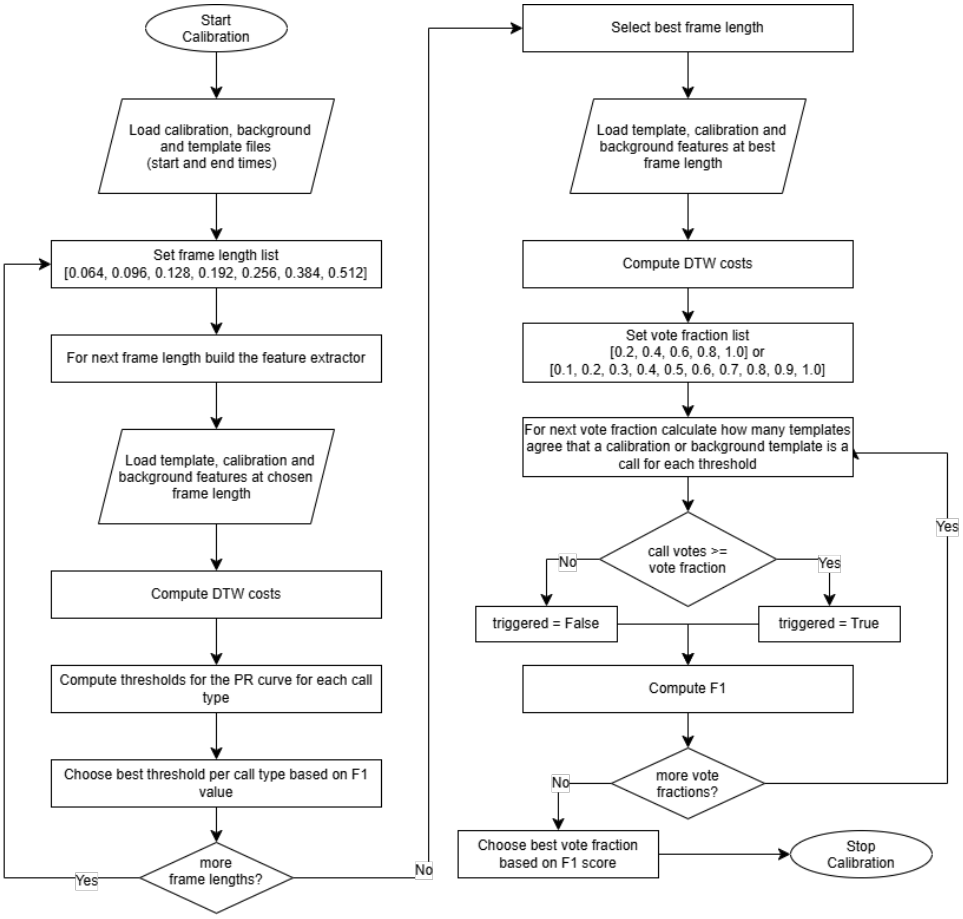


Figure 4.1: Software flow diagram describing the hyperparameter selection procedure.

Three hyperparameters were selected for tuning: feature frame length, call type cost thresholds, and the call vote fraction. Figure 4.1 illustrates the overall execution flow for hyperparameter selection.

As defined in Eq. (2.10), frequency resolution is given by frequency resolution $= \frac{f_s}{L}$. Because Bryde's whale single-tone calls primarily feature a fundamental frequency around 42 Hz and multi-tone calls contain components between 42 and 200 Hz, an upper bound on frequency resolution was set to approximately 10 Hz, corresponding to a frame duration of 100 ms. The lower frequency resolution bound was set to approximately 2 Hz. Although a finer frequency resolution improves spectral bin separation, a 2 Hz resolution requires a frame duration of 500 ms. At a 75% window overlap, this leaves a hop length of 128 ms. Given that the average call duration in the calibration dataset is 246 ms, a 128 ms hop length remains sufficiently short to resolve individual calls. Hees [40] explains that fast Fourier transform (FFT) routines are most computationally efficient for lengths that are powers of two. To maintain computational efficiency, candidate frame lengths were therefore selected as powers of two or multiples thereof: $(0.096, 0.128, 0.192, 0.384, 0.512)$ seconds.

Because the call-type and background calibration templates had to be manually selected due to data scarcity, each calibration template was extended on both sides. This extension ensured that subsequence DTW operated under conditions representative of the evaluation pipeline despite dataset limitations. The total extendd length was configured to exceed the longest evaluation template by at least one frame length on either side. This extra margin was chosen to match the maximum tested frame duration (0.512 s). Each call or background calibration template was centred within this window by adjusting its boundary timestamps accordingly.

For each tested frame length, three decision thresholds were determined (one per call type). These thresholds were determined by evaluating DTW costs between evaluation templates and call type calibration templates, as well as between evaluation templates and background calibration templates. The resulting costs were grouped into call-calibration costs and background-calibration costs. Two hundred candidate thresholds were spaced linearly between the overall minimum and maximum costs across both lists. F1-scores were evaluated across all candidate thresholds for each call type as described in Section 4.6. The threshold yielding the highest f1-score was selected for each respective call type.

For a given frame length, the minimum f1-score among the three call-type thresholds was recorded. The optimal feature frame length was then selected as the one whose minimum threshold f1-score was highest across all tested frame lengths.

Although three distinct thresholds were computed (one per call type), this project focuses exclusively on general call detection rather than classification. This approach was adopted because significant spectral overlap exists among call types. Furthermore, label inconsistencies (arising from differing annotations across human labellers) meant that templates selected for a call type might differ from the ground-truth definitions used in evaluation data. Separate thresholds per call type were nevertheless maintained to account for structural acoustic variations during candidate detection.

In the evaluation pipeline, candidate audio segments are compared against sets of 5 or

10 reference templates per call type. Because templates capture minor structural variations within a call type, individual templates may disagree on whether a candidate segment contains a call. A voting fraction parameter was introduced such that a candidate segment is classified as a call only if a specified fraction of per call type reference templates trigger a detection. Candidate voting fractions were evaluated on the optimal frame length. For 5-template configurations, tested vote fractions were $\{0.2, 0.4, 0.6, 0.8, 1.0\}$; for 10-template configurations, tested fractions were $\{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$.

For each vote fraction, the overall f1-score for general call detection was computed across all call type calibration and background calibration templates. This meant that each group of 5 or 10 templates per call type and its relevant threshold was tested against all call type and background calibration templates. The vote fraction producing the highest overall f1-score was selected.

4.5 Evaluation Procedure

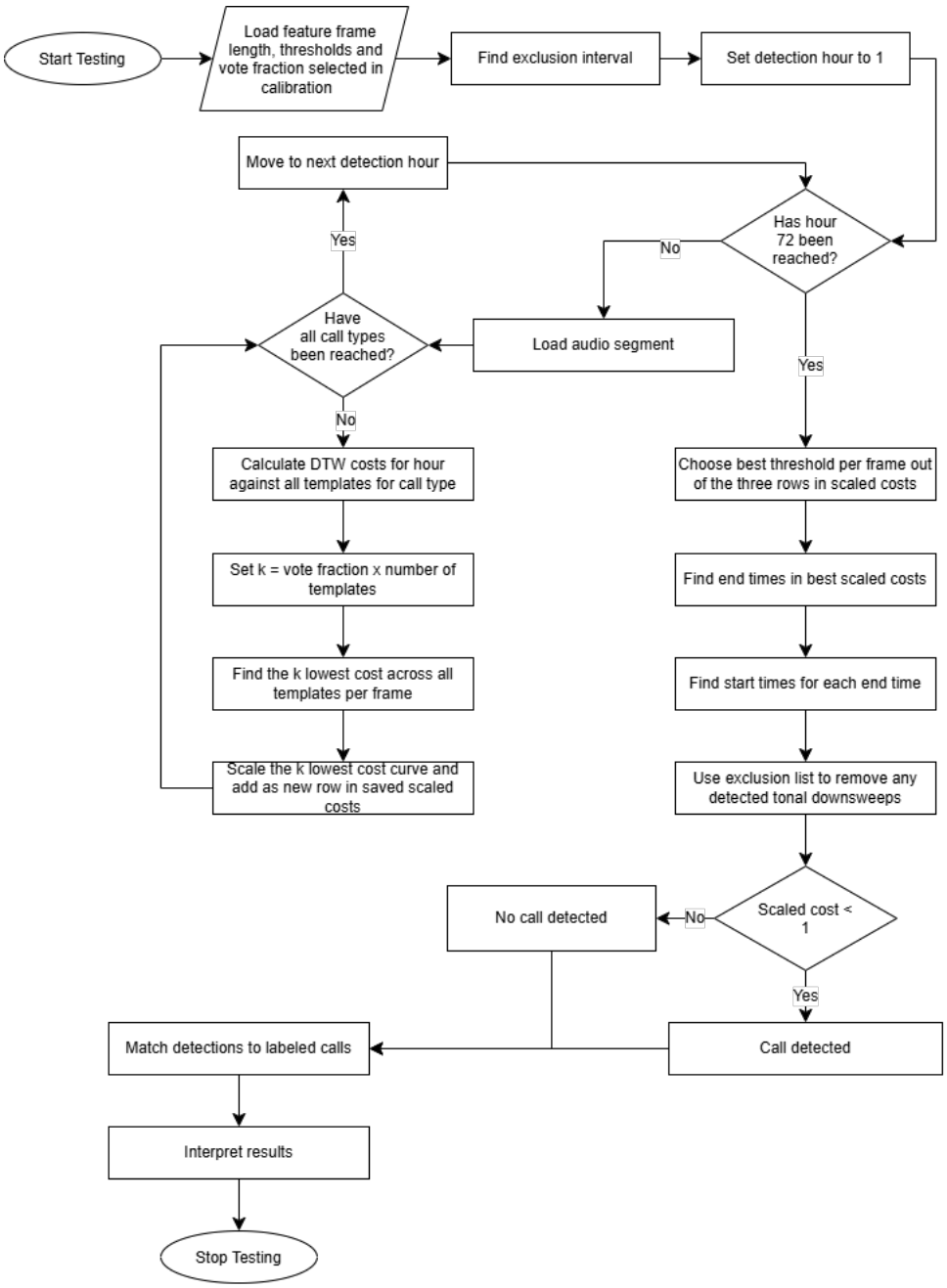


Figure 4.2: Software flow diagram describing the evaluation procedure.

4.5.1 Detection Methodology

The parameters selected during hyperparameter tuning (frame duration, call-type thresholds, and voting fraction) were applied directly during evaluation. The evaluation dataset was the only recording containing tonal downsweeps. Because no calibration threshold was computed for tonal downsweeps, an exclusion index list was maintained for all annotated

tonal downsweeps. To avoid introducing discontinuities in time into the subsequence DTW cost matrix, tonal downsweeps were only excluded after detection rather than omitted prior to processing.

The 72-hour evaluation recording was processed in sequential 1-hour segments to maintain a low memory footprint. The `soundfile` Python library, as documented by Bechtold [41], was used to stream continuous hourly audio blocks without loading the entire dataset into memory. Each hourly segment was downsampled from 8 kHz to 1 kHz using the `scipy.signal.resample_poly` function described in the SciPy documentation [42], and features were extracted as outlined in Section 4.2.1.

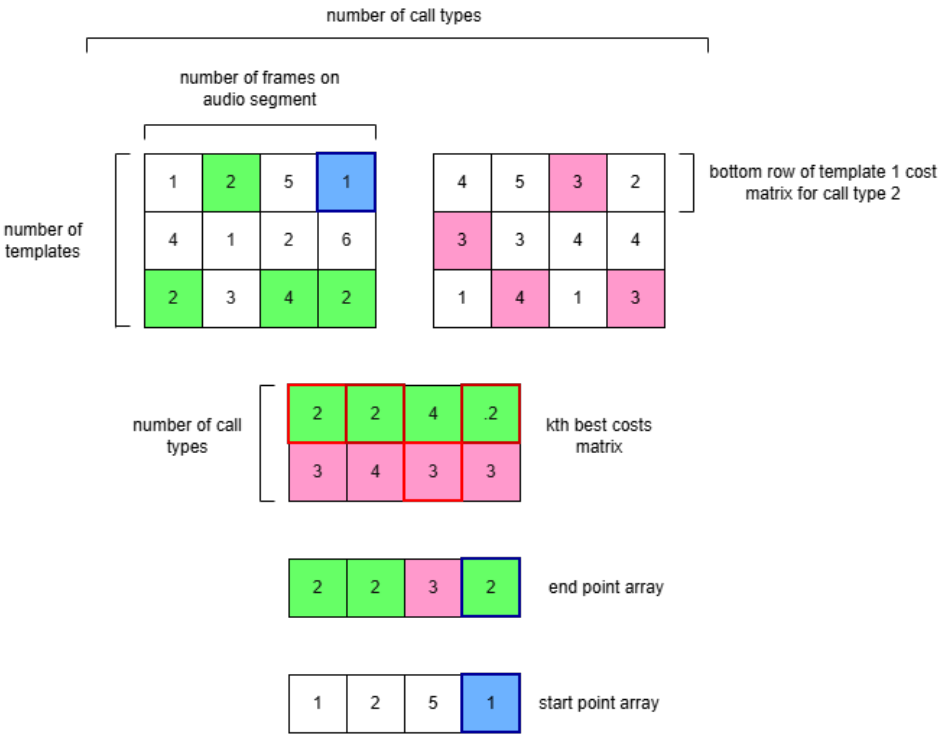


Figure 4.3: Illustration of how k-th lowest costs are extracted and scaled for a sample configuration with three templates per call type, two thresholds, and $k = 2$.

The DTW cost matrix was computed for all templates across each call type, and the bottom row of each cost matrix was extracted and stored, as illustrated in the top diagram of Figure 4.3. The integer parameter k was defined as the vote fraction multiplied by the number of templates per call type. For each frame, the k-th lowest cost across templates was extracted into a single array. Each value in this array was divided by the corresponding call-type threshold. This normalisation created a matrix of scaled k-th costs, where a value less than or equal to 1.0 consistently indicated a valid threshold match across all call types, as shown in the second diagram of Figure 4.3. The minimum scaled cost across call types was then taken to produce a single cost curve per frame.

This unified cost array was evaluated for call detections (third diagram of Figure 4.3).

Evaluating the k-th lowest cost guarantees that the voting fraction requirement is satisfied: if the k-th cost falls below the decision threshold, at least k templates have matched the candidate segment.

Call end times were detected using the `scipy.signal.find_peaks` function, as described in the SciPy documentation [43], on the inverted cost array. In subsequence DTW, candidate matches correspond to local minima along the final row of the accumulated cost matrix. Inverting the cost array ($-$cost) transforms these local cost dips into peaks, allowing candidate end frames with cost ≤ 1.0 to be identified using standard peak-finding algorithms. For each detected end frame, the corresponding call start time was determined. While the end time was identified via the lowest k-th lowest cost among all call type templates, the start time was extracted using the template yielding the absolute lowest cost (from the call type templates of the k-th lowest cost call type) at that end frame. A 10-second audio window preceding the detected end frame was extracted for re-alignment. Because subsequence matches can exceed the baseline template duration, a 10-second window was chosen to accommodate long calls without requiring full recalculation across the entire 1-hour block.

The `librosa.sequence.dtw` function was re-executed on this localized window to compute the step matrix. Backtracking was then performed via `librosa.sequence.dtw_backtracking`, forcing the warp path to originate at the chosen end frame and trace backward to determine the exact start frame. Using the best-matching template (rather than the k-th template) ensures the most accurate start-time boundary estimation.

4.5.2 Matching Methodology

Detected calls were evaluated against labelled calls within the corresponding time period. The `numpy.searchsorted` function, as documented by the NumPy developers [44], was used to find the labelled calls overlapping the detected calls. This was achieved efficiently using four 1D arrays: detected start times, detected end times, labeled start times, and labeled end times.

For each detected call, `numpy.searchsorted` identified the index `lo` in the labeled end-time array where the detected start time would fit sequentially. Similarly, it identified the index `hi` in the labeled start-time array where the detected end time would fit. A condition of `hi` $>$ `lo` indicates that at least one labelled call overlaps with the detected window. If this condition was met, the detection was categorized as a true positive (TP), and the corresponding labelled call was marked as matched. Subsequent detections overlapping an already-matched label were still treated as TPs, as the primary objective of this system is detection rather than specific matches of start and end times.

4.6 Evaluation Metrics

Although receiver operating characteristic (ROC) analysis, as explained by Fawcett [45], is widely used in detection tasks, precision-recall (PR) analysis was chosen due to severe class imbalance between whale calls and background noise. Saito and Rehmsmeier [46] show that ROC curves can present an overly optimistic assessment in heavily imbalanced scenarios, whereas PR curves more accurately reflect performance under class skew. Consequently, precision, recall, and f1-score were selected as primary evaluation metrics. However, because the calibration dataset exhibited a different class imbalance ratio than the evaluation dataset, the calibrated operating point was expected to sit slightly off the theoretical optimum of the evaluation PR curve.

Precision measures the proportion of detected calls that correspond to true labelled calls. A true positive (TP) detection is defined as a detected call window overlapping at least one labelled call, while a false positive (FP) is a detection with no label overlap:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{4.1}$$

Recall measures the proportion of labelled calls successfully detected by the system. In this context, a TP label is defined as a labelled call overlapped by at least one detection, whereas a false negative (FN) is a labelled call that receives no detection overlap:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{4.2}$$

Note that TP definitions differ slightly between precision and recall. Because a single long labelled call may produce multiple detections (or vice versa), any overlap between call detections and labelled calls is treated as a true positive, supporting the system's objective of general call detection.

The f1-score represents the optimal mean of precision and recall:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.3}$$

Hyperparameter selections and model comparisons were evaluated primarily using the maximum f1-score. Additionally, Average precision (AP) was calculated across the continuous PR curve using all-point interpolation, as defined by Padilla et al. [47]:

$$\text{AP}_{\text{all}} = \sum_{n} (R_{n+1} - R_n) P_{\text{interp}}(R_{n+1}) \tag{4.4}$$

where

$$P_{\text{interp}}(R_{n+1}) = \max_{\tilde{R} \geq R_{n+1}} P(\tilde{R}) \tag{4.5}$$

To construct the continuous PR curve for AP calculations, the decision thresholds were swept across a range using a scaling factor α. During initial processing, candidate end points were retained if their scaled cost was below 1.5. A strict cut-off at 1.0 was applied for single-point baseline matching, while retaining detections up to 1.5 ensured sufficient candidate points to generate a full PR curve over 100 linearly spaced threshold steps between 0.01 and 1.5.

Chapter 5

Results

After the final detection and separate analysis of the three different models an overall analysis must be done to determine which of the feature extraction methods was the best as well as whether dynamic time warping (DTW) works overall as a detection algorithm for Bryde's Whales in False Bay.

5.1 Experiment 1: STFT Features

For the short time Fourier transform (STFT) model used for call detection two experiments were conducted. The first considered the results given the use of 5 templates per call type used as the analysis database for both hyperparameter selection and evaluation. The second considered the use of 10 templates per call type, the original 5 with 5 new templates added to increase the database diversity.

Table 5.1: Tabulation of thresholds as well as the respective F1-scores used to choose the best frame length. The average F1-score over all three thresholds per STFT experiment was also given for easier analysis.

Number of Temps	Single Tones		Multi Tones		Burst Tonals		Average F1	Frame Length
	F_1	Threshold	F_1	Threshold	F_1	Threshold		
5	0.682	0.234	0.568	0.256	0.796	0.260	0.682	0.512
10	0.690	0.245	0.663	0.236	0.697	0.230	0.683	0.512

Both the 5 and 10 template experiments had an average F1-score of about 0.68 over all three thresholds. Both chose 0.512 seconds as the best frame length. The individual F1-scores for the 5 template experiment were less consistent across the three thresholds than for the 10 template experiment. This can be seen in Table 5.1 as for the 10 template experiment the F1-score consistently stays between 0.66 and 0.70. For the 5 template experiment the change in F1-score between thresholds was much larger staying between

0.56 and 0.80. The absolute highest F1-score for a single threshold was 0.796 for the burst-tonal call type of the 5 template experiment.

Table 5.2: Vote frac results for STFT for 5 and 10 templates

Number of Temps	Precision	Recall	F1	Vote Fraction
5	0.964	0.900	0.931	0.6
10	0.967	0.967	0.967	0.4

The precision, recall and F1 scores for the two STFT experiments during vote fraction selection can be seen in Table 5.2. The vote fraction for the 5 template experiment was 0.6 and for the 10 template experiment it was 0.4. This means a larger percentage of the 5 template experiment votes had to agree on a call than for the 10 template experiment.

In terms of the precision, recall and F1-score the 10 template experiment performed the best for its chosen vote fraction. The final F1-score was 0.967. This shows that for the chosen thresholds at the best possible feature frame length the general detection of a call when compared to all 30 call type templates, 10 for each call type, performed well under the chosen vote fraction. There is an expected drop in precision, recall and F1-score for detection of evaluation data. This drop is due to the vote fraction test being applied using the same calibration templates used to choose the three thresholds and feature frame length. The reuse of calibration templates is due to data scarcity and thus gives an overly optimistic performance result of the model which should only be considered in choosing the vote fraction and not in determining the actual model performance.

Table 5.3: Final test results for STFT for 5 and 10 templates

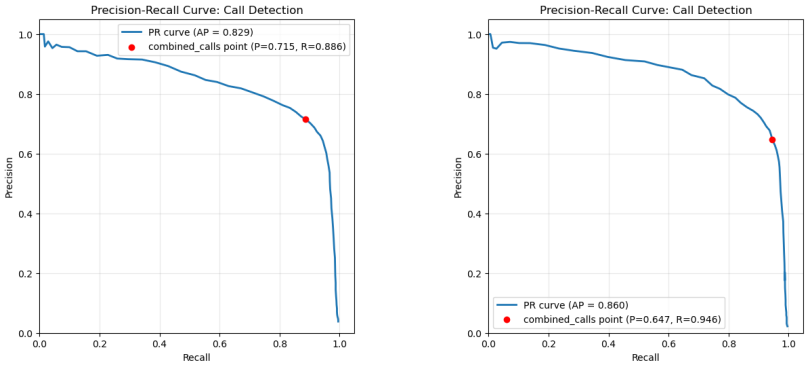
Number of Temps	Precision	Recall	F1	Average Precision
5	0.715	0.886	0.792	0.829
10	0.647	0.946	0.769	0.860

In terms of precision, as seen in Table 5.3, the 5 template experiment performed the best with a difference in precision between the two experiments of 0.068 and the highest being 0.715. However, in terms of recall the 10 template experiment performed the best with a 0.06 difference and the highest being 0.946.

Since the precision difference between the two experiments was the largest the 5 template experiment outperformed the 10 template experiment. This can also be seen in the final F1 score with the highest being 0.792 for the 5 template experiment.

By looking at Figures 5.1a and 5.1b it can be seen that both experiments performed well in terms of the selection of the hyperparameters as both marked precision and recall points

are in the optimal position of their respective curve. Therefore despite the slightly better numerical performance both STFT experiments performed well in terms of hyperparameter selection.



(a) 5 templates **(b)** 10 templates

Figure 5.1: The PR curves for STFT

By again looking at Table 5.3 and Figures 5.1a and 5.1b, it can be seen that in terms of average precision (AP) and therefore the overall precision recall (PR) curve of the model the 10 template experiment outperformed the 5 template experiment with the highest AP equal to 0.860.

5.2 Experiment 2: MFCC Features

For the mel-frequency cepstral coefficient (MFCC) model three main experiments were considered. The first experiment used the MFCC output with no added derivatives for the feature. The second experiment used the original 13 feature coefficients from experiment 1 and added 13 more coefficients in the form of the derivatives of the original 13. The third and final experiment considered the 26 coefficients used in the second experiment and added another 13 in the form of the second derivatives of the original 13 MFCC output coefficients.

For each experiment two sub-experiments were performed. The first was performed using 5 templates and the second using 10 templates.

Table 5.4: Calibration variables chosen via highest F1 for 5 templates and 10 templates

	Number of Temps	Single Tones		Multi Tones		Burst Tonals		Average F1	Frame Length
		F_1	Threshold	F_1	Threshold	F_1	Threshold		
MFCC	5	0.718	16.394	0.675	22.688	0.500	19.483	0.631	0.384
	10	0.784	17.507	0.402	19.553	0.472	21.288	0.553	0.256
MFCC deriv1	5	0.683	15.610	0.659	23.440	0.500	20.337	0.614	0.384
	10	0.785	18.815	0.394	22.548	0.425	20.569	0.535	0.256
MFCC deriv2	5	0.758	18.199	0.659	24.977	0.518	21.595	0.600	0.384
	10	0.806	19.642	0.408	22.356	0.430	22.836	0.548	0.256

As seen in Table 5.4 all three experiments for the MFCC model chose the same frame length for the 5 and 10 template sub-experiments. These values were 0.384 and 0.256 seconds for the 5 and 10 template sub-experiments respectively.

For all three experiments the 5 template sub-experiment achieved a higher average F1-score over all three thresholds. In terms of average F1 score over all three thresholds the MFCC experiment without derivatives performed the best. The highest average F1-score was for the 5 template sub-experiment with a score of 0.631. The second highest average F1-score was for the experiment where the first derivatives were added with a score of 0.614.

In terms of the 10 template sub-experiments the experiment only using the first 13 MFCC output coefficients performed the best in terms of average F1-score with a score of 0.553. The second best F1-score across the three 10 template sub-experiments was for the third experiment using all 39 coefficients. This score was 0.548.

For all three experiments and their respective sub-experiments the single-tone call type performed the best in terms of F1-score for its chosen threshold. The highest F1-score was 0.806 for the 10 template sub-experiment of the third experiment containing all 39 coefficients.

Table 5.5: Final vote results for 5 and 10 templates

	Number of Temps	Precision	Recall	F_1	Vote Fraction
MFCC	5	1.000	0.900	0.947	0.6
	10	0.935	0.967	0.951	0.5
MFCC deriv1	5	0.935	0.967	0.951	0.4
	10	0.964	0.900	0.931	0.6
MFCC deriv2	5	0.906	0.967	0.935	0.4
	10	1.000	0.900	0.947	0.6

In terms of the F1-score for the choice of vote fraction, as seen in Table 5.5, the highest score of 0.951 was shared by the 10 template sub-experiment for the MFCC experiment without derivatives and the 5 template sub-experiment for MFCC experiment with the first derivatives added.

The second highest F1-score of 0.947 was shared by the 5 template sub-experiment for the MFCC experiment without derivatives and the 10 template sub-experiment of the MFCC experiment with two derivatives.

Considering that the MFCC experiment with no derivatives held both the first and second highest F1-scores this model was seen to have performed best in terms of overall call detection at the chosen hyperparameter settings for the calibration data. This performance can only be considered in terms of the hyperparameter choices and not in terms of the overall performance of the model due to the use of the same calibration templates for all hyperparameter selection choices.

The vote fractions chosen for the MFCC experiments with the first and second derivatives stayed constant in terms of number of templates. The 5 template sub-experiments chose a vote fraction of 0.4 and the 10 template sub-experiments chose 0.6. The MFCC experiment without derivatives had different vote fractions with the 5 template sub-experiment choosing 0.6 and the 10 template sub-experiment choosing 0.5.

Table 5.6: Final test results for 5 and 10 templates

	Number of Temps	Precision	Recall	F_1	Average Precision
MFCC	5	0.582	0.714	0.641	0.642
	10	0.535	0.817	0.647	0.684
MFCC deriv1	5	0.495	0.799	0.611	0.632
	10	0.468	0.809	0.593	0.602
MFCC deriv2	5	0.354	0.880	0.505	0.614
	10	0.505	0.770	0.610	0.628

For the MFCC experiment with no derivatives the 5 template sub-experiment performed the best for precision. The 10 template sub-experiment performed best in terms of recall and F1 score. The highest values for precision, recall and F1 score respectively were 0.582, 0.817 and 0.647.

For the MFCC experiment with the first derivatives added the 5 template sub-experiment performed best in precision and F1 score while the 10 template sub-experiment performed best for recall. The highest values for precision, recall and F1 score respectively were 0.495, 0.809 and 0.611.

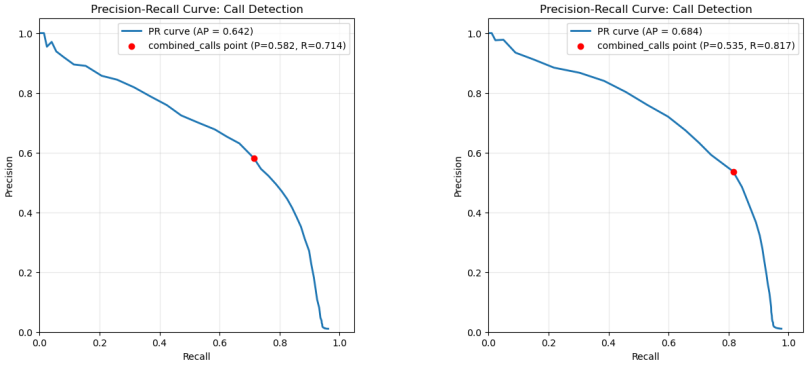
For the MFCC experiment with the second derivatives added the 5 template sub-experiment performed best for recall. The 10 template sub-experiment performed best for

precision and F1 score. The highest values for precision, recall and F1 score respectively were 0.505, 0.880 and 0.610.

Overall in terms of F1-score the 10 template sub-experiment for the MFCC experiment with no derivatives performed the best. By looking at the values in Table 5.6 it can be seen that the highest overall hyperparameter selection performance is characterized by averaging out the precision and recall and not by getting the highest value in either category.

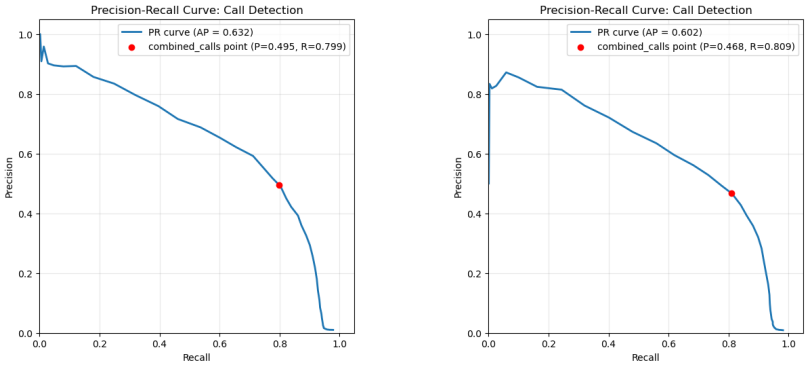
In terms of average precision (AP) and precision and recall (PR) curve the MFCC experiment with no derivatives outperformed all other MFCC experiments with a maximum AP of 0.684 for the 10 template sub-experiment. This can be seen by looking at both Table 5.6 and Figures 5.2, 5.3 and 5.4.

The 10 template sub-experiment for the MFCC experiment with no derivatives is the overall best performer for the MFCC models.



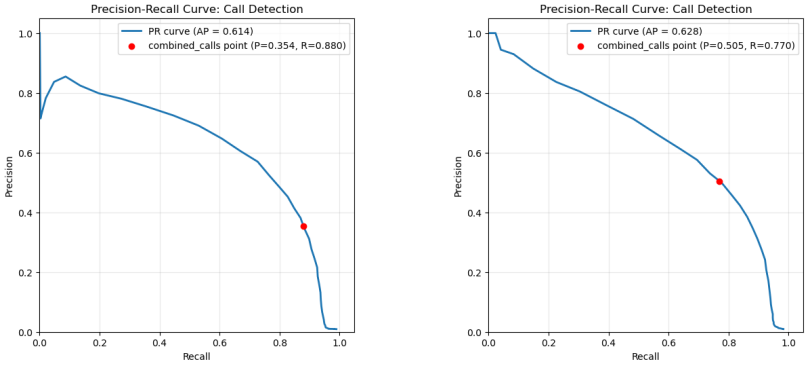
(a) 5 templates **(b)** 10 templates

Figure 5.2: The PR curves for MFCC



(a) 5 templates **(b)** 10 templates

Figure 5.3: The PR curves for MFCC with the first derivative added



(a) 5 templates **(b)** 10 templates

Figure 5.4: The PR curves for MFCC with the first and second derivatives added

5.3 Experiment 3: LPCC Features

For the linear predictive coding coefficients (LPCC) model two experiments were conducted. The first considered the use of 5 templates and the second considered the use of 10 templates for the call type database for hyperparameter selection and evaluation.

Table 5.7: Calibration variables chosen via highest F1 for LPC for 5 templates and 10 templates

Number of Temps	Single Tones		Multi Tones		Burst Tonals		Average F1	Frame Length
	F_1	Threshold	F_1	Threshold	F_1	Threshold		
5	0.863	0.344	0.744	0.526	0.632	0.449	0.746	0.256
10	0.882	0.336	0.605	0.364	0.686	0.427	0.724	0.256

Both the 5 and 10 template experiments for the LPC model chose 0.256 seconds as the frame length. This can be seen in Table 5.7.

The 5 template model had the highest average F1-score of 0.746. This score was 0.022 higher than the 10 template experiment F1-score. There was no consistency in terms of F1-scores across the three call type thresholds for either the 5 or 10 template experiment.

Table 5.8: Vote frac results for LPC for 5 and 10 templates

Number of Temps	Precision	Recall	F1	Vote Fraction
5	0.968	1.000	0.984	0.4
10	0.968	1.000	0.984	0.5

While the 5 and 10 template experiments for LPC each chose a different vote fraction, in terms of precision, recall and F1 score they performed equally well. The chosen vote fraction for each was 0.4 and 0.5 for the 5 and 10 template experiments respectively. The F1-score for both experiments was 0.984. This can be seen in Table 5.8.

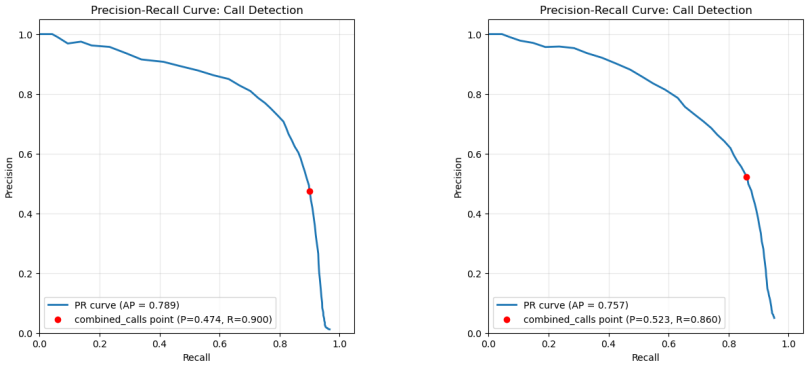
This F1-score carries no weight in terms of the actual performance of the model when tested against evaluation data.

Table 5.9: Final test results for LPC for 5 and 10 templates

Number of Temps	Precision	Recall	F1	Average Precision
5	0.474	0.900	0.621	0.789
10	0.523	0.860	0.650	0.757

As seen in Table 5.9, the 5 template experiment performed better in terms of recall. The 10 template experiment performed best in precision and F1 score. The highest precision, recall and F1-score were 0.523, 0.900 and 0.650 respectively.

This means that the 10 template experiment performed better in terms of chosen thresholds. This can also be seen in Figures 5.5a and 5.5b where the marked precision and recall point is better matched to the precision recall (PR) curve for the 10 template experiment.



(a) 5 templates **(b)** 10 templates

Figure 5.5: The PR curves for LPC

By considering Table 5.9 and Figure 5.5a and 5.5b, it can be seen that in terms of average precision (AP) and PR curve shape the 5 template experiment performed better with an AP value of 0.789.

Even though the 10 template experiment gave a better PR point due to chosen thresholds the 5 template experiment for the LPC model performed best overall in terms of Bryde's Whale call detections due to the PR curve shape and AP value.

5.4 General Observations

The lack of data and human error meant that a maximum of 10 templates per call type could be used for call type calibration templates. It also caused a bias in the choices of templates to train on as selecting suitably representative templates per call type became difficult. The selection of background calibration templates meant that the class imbalance between calls and background noise differed between hyperparameter selection and evaluation. The selection of templates and not a continuous audio file for hyperparameter selection also meant that subsequence dynamic time warping (DTW) did not necessarily act within the same conditions as evaluation thus creating another difference between the two datasets.

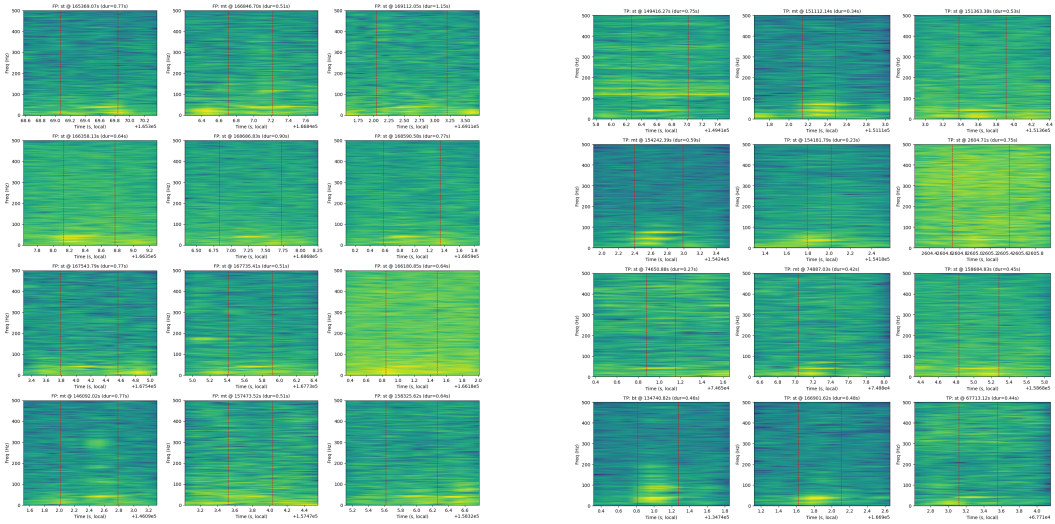
Considering the low availability of well labeled data to use for the hyperparameter selection stage of this project as well as human error, which was introduced in manual selection of calibration templates, the varying performance of the calibrated thresholds is justified. This varying performance can be seen in Tables 5.1, 5.4 and 5.7.

Table 5.10: Overall highest results for F1 and AP per model

Feature extraction method	F1		AP	
	Highest value	Number of Templates	Highest value	Number of Templates
STFT	0.792	5	0.860	10
MFCC	0.647	10	0.684	10
LPC	0.650	10	0.789	5

By looking at Table 5.10 it can be seen that the short time Fourier transform (STFT) model outperforms the other models in terms of both F1-score and average precision (AP). This meant that both in terms of the models ability to select hyperparameters and in terms of the efficiency of the detection pipeline the STFT model outperformed the other two models. The linear predictive coding coefficient (LPCC) model comes second, but in terms of precision it is not much better than the mel-frequency cepstral coefficient (MFCC) model.

When looking at printed spectrograms of false positives (FP) detected by the STFT model it was found that for both experiments for the best performing model many of these FP calls were actually unlabeled calls. This means that the actual precision of the model may be higher than the reported amounts. This may be due to the human error introduced in call labeling, since the known labeled calls were found by previous projects and were therefore not necessarily a complete list. Examples of FP calls compared to similar true positive (TP) calls can be seen in Figure 5.6

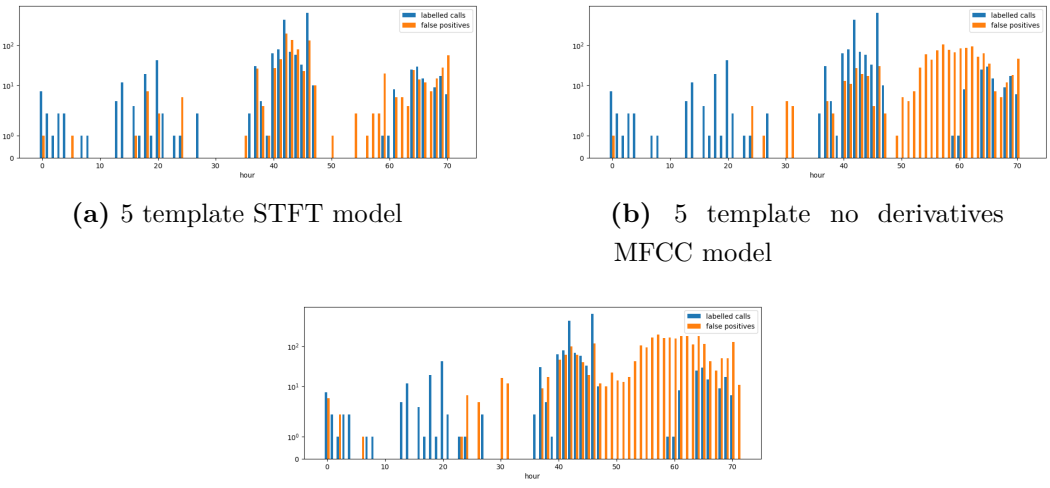


(a) False Positive samples

(b) True Positive samples

Figure 5.6: Examples of random samples of FP and TP for which some FP could be unlabeled calls for the 10 template experiment for the STFT model.

Another interesting observation across all three detection models was that most of the FP calls were found within the second half of the 72 hour audio file used for the evaluation stage of this project. This implies a possible link between model performance and the signal-to-noise ratio (SNR) of the section of audio considered. This observation can be seen in Figure 5.7



(a) 5 template STFT model

(b) 5 template no derivatives MFCC model

(c) 5 template LPCC model

Figure 5.7: This shows for each model where FP calls were detected in relation to labeled calls over the entire 72 hour audio file.

Considering the good performance of the model as well as the unlabeled TP calls, the STFT feature extraction is the best method to use for call detection with the DTW

algorithm and the algorithm is a valid choice as detection method for Bryde's Whales within the False Bay region.

Chapter 6

Conclusion

References

[1] K. van Waerebeek, A. N. Baker, F. Félix, J. Gedamke, M. Iñiguez, G. P. Sanino, E. Secchi, D. Sutaria, A. van Helden, and Y. Wang, "Vessel collisions with small cetaceans worldwide and with large whales in the southern hemisphere; building a standardized database," 2006, https://www.vliz.be/imisdocs/publications/243193.pdf.

[2] A. Faustmann, F. W. Shabangu, and J. D. J. J. Versfeld, "Potential vocalizations of bryde's whales in false bay, south africa," 2024, https://repository.up.ac.za/server/api/core/bitstreams/57ba6a31-7aca-4acb-b9b6-60884695c964/content.

[3] S. Betz, K. Bohnsack, A. R. Callahan, L. E. Campbell, S. E. Green, and K. M. Labrum, "Reducing the risk of vessel strikes to endangered whales in the santa barbara channel," 2011, https://nmssanctuaries.blob.core.windows.net/sanctuaries-prod/media/archive/protect/shipstrike/pdfs/bren.pdf.

[4] V. B. Deecke, J. K. B. Ford, and P. Spong, "Call diversity in the north pacific killer whale populations: implications for dialect evolution and population history," *Animal Behaviour*, 2000, https://www.sciencedirect.com/science/article/abs/pii/S0003347211005537.

[5] M. A. McDonald, S. L. Mesnick, and J. A. Hildebrand, "The geographic variation of blue whale song," 2003, https://www.cetus.ucsd.edu/docs/reports/McDonaldIWC-2003.pdf.

[6] J. C. Brown, A. Hodgins-Davis, and P. J. O. Miller, "Classification of vocalizations of killer whales using dynamic time warping," 2006, https://pubs.aip.org/asa/jasa/article/119/3/EL34/850621.

[7] J. Herreilers, C. Jacobs, and T. Niesler, "Low-resource keyword spotting using contrastively trained transformer acoustic word embeddings," 2025, https://arxiv.org/pdf/2506.17690.

[8] E. M. Oleson, J. Barlow, J. Gordon, S. Rankin, and J. A. Hildebrand, "Low frequency calls of bryde's whales," 2003, https://digitalcommons.unl.edu/cgi/viewcontent.cgi?referer=&httpsredir=1&article=1297&context=usdeptcommercepub.

[9] M. Müller and F. Zalkow, "Fundamentals of music processing: Dynamic time warping (dtw)," 2015, https://www.audiolabs-erlangen.de/resources/MIR/FMP/C3/C3S2_DTWbasic.html.

[10] H. Kamper, "Dynamic time warping (dtw)," 2021, https://www.youtube.com/playlist?list=PLmZlBIcArwhMJoGk5zpiRlkaHUqy5dLzL.

[11] R. Schwaber-Cohen, "Vector similarity explained," 2023, https://www.pinecone.io/learn/vector-similarity/.

[12] T. S. community, "scipy.spatian.distnace," 2008, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cdist.html.

[13] E. Alizadeh, "An illustrative explanation to dynamic time warping," 2020, https://towardsdatascience.com/an-illustrative-introduction-to-dynamic-time-warping-36aa98513b98/.

[14] M. Müller and F. Zalkow, "Fundamentals of music processing: Subsequence dtw," 2015, https://www.audiolabs-erlangen.de/resources/MIR/FMP/C7/C7S2_SubsequenceDTW.html.

[15] D. J. J. V. Atinde M. Usman, Olayinka O. Ocundile, "Review of automatic detection and classification techniques for cetacean vocalization," 2020, https://scholar.sun.ac.za/server/api/core/bitstreams/6e4a861d-74eb-4d2d-b81b-353743cc06b8/content.

[16] H. Kamper, "Short-time fourier transforms (stft)," 2026, https://www.youtube.com/watch?v=tQeDi2qEW-s&list=PLmZlBIcArwhMrUZ_njMmlO_T8gAWHihNS&index=26.

[17] ——, "Introduction to speech features," 2021, https://www.youtube.com/playlist?list=PLmZlBIcArwhN8nFJ8VL1jLM2Qe7YCcmAb.

[18] T. Singh, "Mfcc's made easy," 2019, https://medium.com/@tanveer9812/mfccs-made-easy-7ef383006040.

[19] DataMListic, "Spectral features - deltas and delta-deltas explained," 2023, https://www.youtube.com/watch?v=zxEnuPolylY.

[20] ——, "Mel frequency cepstral coefficients (mfcc) explained," 2023, https://www.youtube.com/watch?v=SJo7vPgRlBQ.

[21] S. A. Majeed, H. Husain, S. A. Samad, and T. F. Idbeaa, "Mel frequency cepstral coefficients (mfcc) feature extraction enhancement in the application of speech recognition: A comparison study," 2015, https://www.jatit.org/volumes/Vol79No1/5Vol79No1.pdf.

[22] L. Rabiner and B.-H. J. Juang, *Fundamentals of Speech Recognition*. Prentice Hall, 1993.

[23] J. C. Brown and P. J. O. Miller, "Automatic classification of killer whale vocalizations using dynamic time warping," 2007, https://pubs.aip.org/asa/jasa/article-abstract/122/2/1201/699104/Automatic-classification-of-killer-whale?redirectedFrom=fulltext&casa_token=OzamuY0etVkAAAAA:VNGCuA1SmTnQEYgVRN2KuKcJvbDF79bLT2VkmggZSIUc2qnhYfNt_Yo7BleyPL4ffIKE3CTopQ.

[24] O. O. Ogundile and D. J. J. Versfeld, "Analysis of template-based detection algorithms for inshore bryde's whale short pulse calls," 2020, https://ieeexplore.ieee.org/document/8957435.

[25] X. Mouy, M. Bahoura, and Y. Simard, "Automatic recognition of fin and blue whale calls for real-time monitoring in the st. lawrence," 2009, https://pubs.aip.org/asa/jasa/article-abstract/126/6/2918/627068/Automatic-recognition-of-fin-and-blue-whale-calls?redirectedFrom=fulltext&casa_token=_uPiaa9QXCsAAAAA:Ge3i-ujMkyY8PsWGIQ4iGgW0FRxsPD0VtXtwuCMONoJvGUJFQAuBTQu8Iq7TWFSif-LMp-Eos

[26] O. O. Ogundile, A. Usman, O. Babalola, and D. Versfeld, "A hidden markov model with selective time domain feature extraction to detect inshore bryde's whale short pulse calls," 2020, https://www.sciencedirect.com/science/article/pii/S1574954120300376?casa_token=6Nq5H72ZrPQAAAAA:X0axwemDJS8TgltJ62x5518DbviKiMtXRjb7HX_TJkOnpH1IpbmheAeQcZnjfbrx6TNF-oP_Z7o.

[27] Y. Permanasari, E. H. Harahap, and E. P. Ali, "Speech recognition using dynamic time warping (dtw)," 2019, https://iopscience.iop.org/article/10.1088/1742-6596/1366/1/012091/meta.

[28] A. Faustmann, "Potential vocalizations of bryde's whales in false bay, south africa," 2024, https://scholardata.sun.ac.za/articles/dataset/An_Analysis_of_Potential_Low_Frequency_Vocalizations_of_Southern_African_Bryde_s_Whales_/25108016?file=44351747.

[29] G. Tonitz, C. M. Geldenhuys, and T. R. Niesler, "A perch out of water: Few-shot detection of bryde's whale vocalisations in false bay," 2026, https://arxiv.org/abs/2610.29572.

[30] B. Meddins, "The nyquist frequency," 2000, https://www.sciencedirect.com/topics/computer-science/nyquist-frequency.

[31] A. M. Learning, "Windowing functions explained," 2026, https://apxml.com/courses/introduction-to-speech-recognition/chapter-2-processing-audio-signals/windowing-functions-explained.

[32] librosa development team, "librosa.feature.mfcc," 2026, https://librosa.org/doc/main/api/generated/librosa.feature.mfcc.html.

[33] T. S. community, "dct," 2008, https://docs.scipy.org/doc/scipy/reference/generated/scipy.fft.dct.html.

[34] M. A. Roch, A. Širović, and S. Baumann-Pickering, "Detection, classification, and localization of cetaceans by groups at the scripps institution of oceanography and san diego state university," 2013, https://www.cetus.ucsd.edu/docs/publications/RochDCL2013.pdf.

[35] librosa development team, "librosa.feature.delta," 2023, https://librosa.org/doc/dev/api/generated/librosa.feature.delta.html.

[36] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in python," 2015, https://proceedings.scipy.org/articles/Majora-7b98e3ed-003.pdf.

[37] librosa development team, "librosa.util.frame," 2025, https://librosa.org/doc/0.11.0/generated/librosa.util.frame.html.

[38] ——, "librosa.lpc," 2025, https://librosa.org/doc/0.11.0/generated/librosa.lpc.html.

[39] ——, "librosa.sequence.dtw," 2026, https://librosa.org/doc/0.11.0/generated/librosa.sequence.dtw.html.

[40] P. Hees, "The fast fourier transform, how and why it works," 2026, https://research.ibm.com/blog/the-fast-fourier-transform-how-and-why-it-works.

[41] B. Bechtold, "soundfile 0.14.0," 2026, https://pypi.org/project/soundfile/.

[42] T. S. community, "resample_poly," 2008, http://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.resample_poly.html.

[43] ——, "find_peaks," 2008, https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.find_peaks.html.

[44] N. Developers, "numpy.searchsorted," 2026, https://numpy.org/doc/stable/reference/generated/numpy.searchsorted.html.

[45] T. Fawcett, "An introduction to roc analysis," 2005, https://www.sciencedirect.com/science/article/pii/S016786550500303X.

[46] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets," 2015, https://www.researchgate.net/publication/273155496_The_Precision-Recall_Plot_Is_More_Informative_than_the_ROC_Plot_When_Evaluating_Binary_Classifiers_on_Imbalanced_Datasets.

[47] R. Padilla, S. L. Netto, and E. A. B. da Silva, "A survey on performance metrics forobject-detection algorithms," 2020, https://www.researchgate.net/publication/343194514_A_Survey_on_Performance_Metrics_for_Object-Detection_Algorithms.

Appendix A

Appendix title goes here